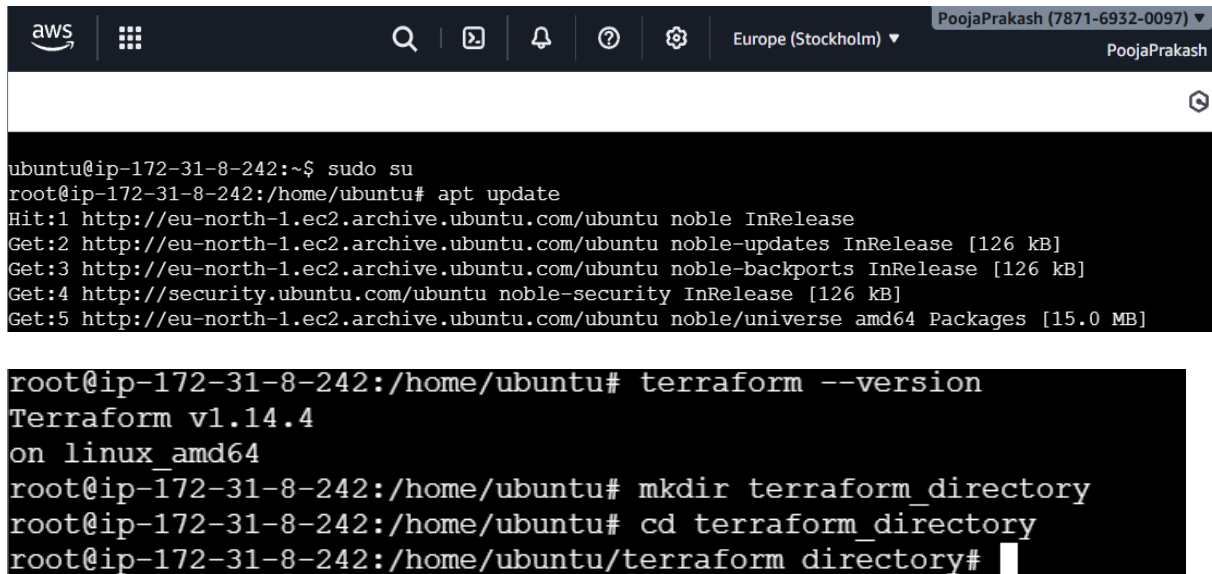


# Terraform Practical

Name: Pooja Prakash Belgaumkar

## 1. AWS EC2 provisioning using Terraform

- Launch an EC2 instance, connect to it and install Terraform on the EC2 machine.

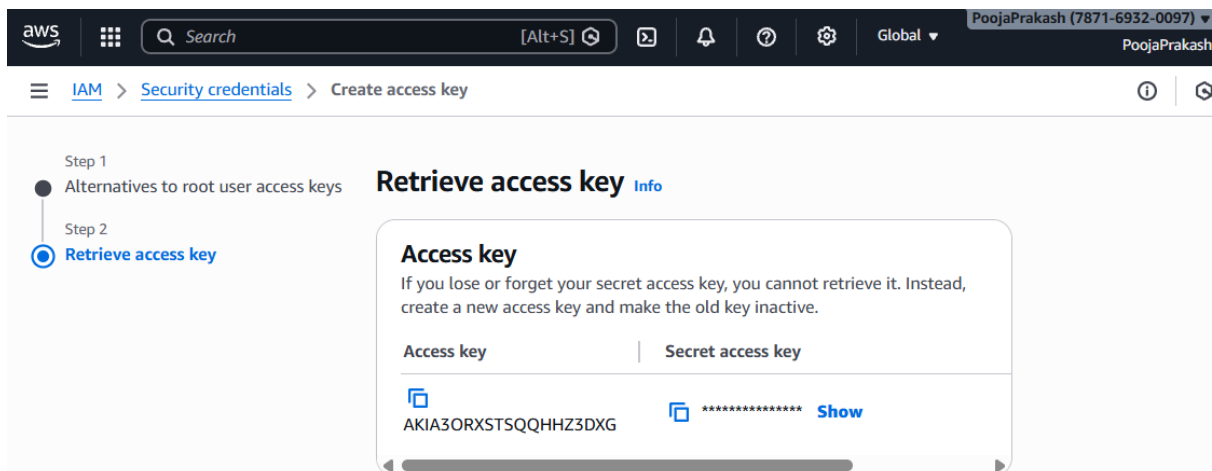


The screenshot shows the AWS Management Console interface. The top navigation bar includes the AWS logo, a search bar, and various utility icons. The user's profile is 'PoojaPrakash (7871-6932-0097)' in the 'Europe (Stockholm)' region. The main content area displays a terminal window with the following commands and output:

```
ubuntu@ip-172-31-8-242:~$ sudo su
root@ip-172-31-8-242:/home/ubuntu# apt update
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]

root@ip-172-31-8-242:/home/ubuntu# terraform --version
Terraform v1.14.4
on linux_amd64
root@ip-172-31-8-242:/home/ubuntu# mkdir terraform_directory
root@ip-172-31-8-242:/home/ubuntu# cd terraform_directory
root@ip-172-31-8-242:/home/ubuntu/terraform_directory#
```

- Configure the AWS account by creating a configuration.tf file and providing the Access Key, Secret Key and region.



```

provider "aws" {
  region      = "eu-north-1"
  access_key  = "AKIA3ORXSTS..."
  secret_key  = "UoM2mptMTaSsrGu9tL+qpb..."
}
~
~
~
~

```

- Run the *terraform init* command to initialize Terraform in the project directory.

```

root@ip-172-31-8-242:/home/ubuntu/terraform_directory# vi configuration.tf
root@ip-172-31-8-242:/home/ubuntu/terraform_directory# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.31.0...
- Installed hashicorp/aws v6.31.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

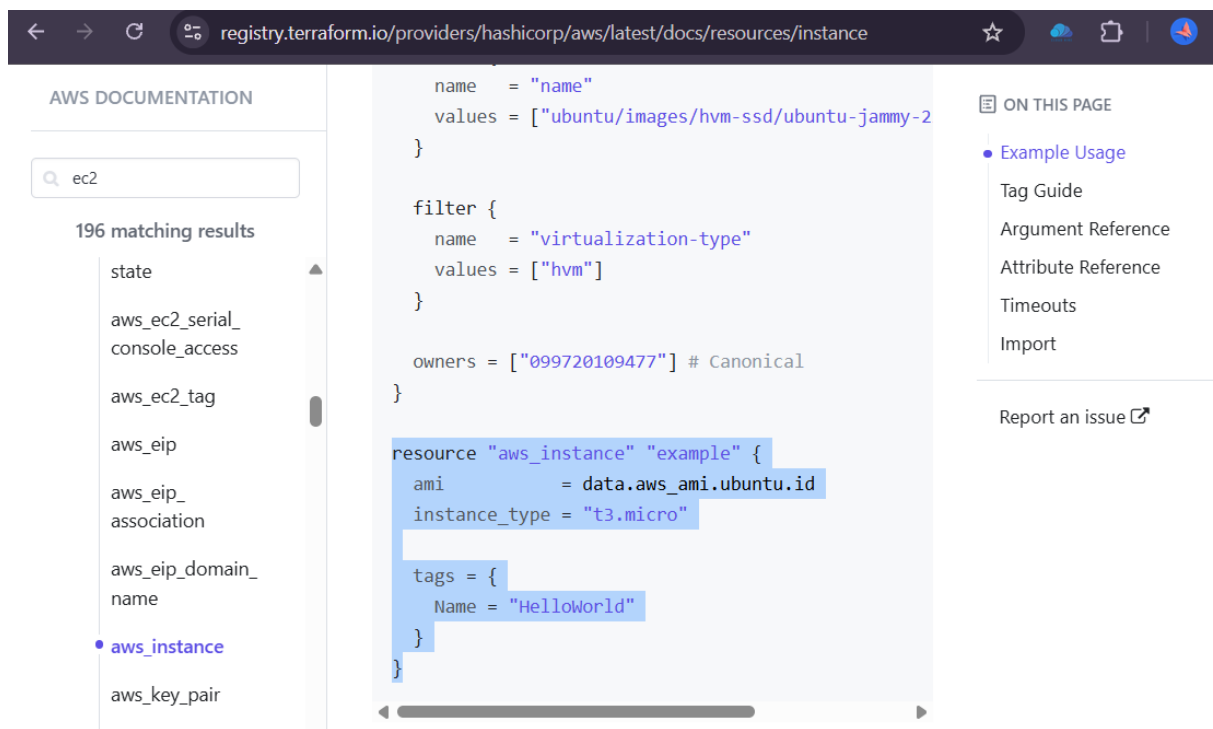
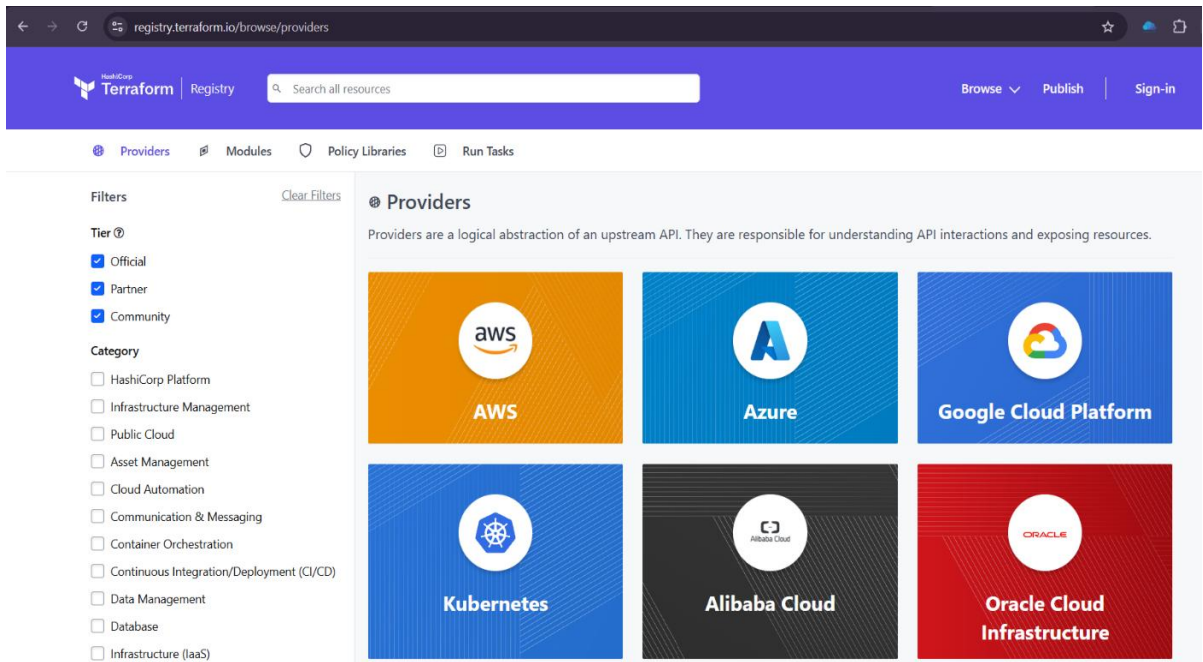
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
root@ip-172-31-8-242:/home/ubuntu/terraform_directory#

```

- Refer to the official Terraform Registry ([registry.terraform.io](https://registry.terraform.io)) to understand the required HCL syntax and resource configuration. Select the appropriate provider based on the cloud platform being used (AWS in this case).
- Write the infrastructure configuration using HCL (HashiCorp Configuration Language) in the .tf files based on the official Terraform documentation.



- Create an AWS instance resource named **MyFirstInstance** with the identifier **id01** and specify the instance type as **t3.micro**.

```
resource "aws_instance" "id01" {
  ami           = "ami-073130f74f5ffb161"
  instance_type = "t3.micro"

  tags = {
    Name = "MyFirstInstance"
  }
}
```

- Run the *terraform plan* command to generate an execution plan that shows what Terraform will create when the *terraform apply* command is executed.

```
root@ip-172-31-8-242:/home/ubuntu/terraform_directory# vi ec2.tf
root@ip-172-31-8-242:/home/ubuntu/terraform_directory# terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.id01 will be created
+ resource "aws_instance" "id01" {
  + ami           = "ami-073130f74f5ffb161"
  + arn           = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone           = (known after apply)
  + disable_api_stop           = (known after apply)
  + disable_api_termination    = (known after apply)
  + ebs_optimized              = (known after apply)
  + enable_primary_ipv6        = (known after apply)
  + force_destroy              = false
  + get_password_data          = false
  + host_id                   = (known after apply)
  + host_resource_group_arn    = (known after apply)
  + iam_instance_profile       = (known after apply)
  + id                       = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle         = (known after apply)
  + instance_state             = (known after apply)
```

- Finally, run the *terraform apply* command to create the required infrastructure defined in the .tf files.

```
root@ip-172-31-8-242:/home/ubuntu/terraform_directory# terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
+ create
```

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.id01: Creating...
aws_instance.id01: Still creating... [00m10s elapsed]
aws_instance.id01: Creation complete after 12s [id=i-05bea27f12bdf00d7]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
root@ip-172-31-8-242:/home/ubuntu/terraform directory#

```

Instances (1) [Info](#) Last updated less than a minute ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

[Running](#)

[Instance ID = i-05bea27f12bdf00d7](#) [Clear filters](#)

| <input type="checkbox"/> | Name <a href="#">↗</a> | Instance ID         | Instance state <a href="#">↕</a>            | Instance type <a href="#">↕</a> | Status check      | Alarm status                |
|--------------------------|------------------------|---------------------|---------------------------------------------|---------------------------------|-------------------|-----------------------------|
| <input type="checkbox"/> | MyFirstInstance        | i-05bea27f12bdf00d7 | Running <a href="#">🔍</a> <a href="#">🔍</a> | t3.micro                        | 3/3 checks passed | <a href="#">View alarms</a> |

- To delete the created resources, run the *terraform destroy* command.

```

root@ip-172-31-8-242:/home/ubuntu/terraform_directory# terraform destroy
aws_instance.id01: Refreshing state... [id=i-05bea27f12bdf00d7]

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# aws_instance.id01 will be destroyed
- resource "aws_instance" "id01" {

```

```

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.id01: Destroying... [id=i-05bea27f12bdf00d7]
aws_instance.id01: Still destroying... [id=i-05bea27f12bdf00d7, 00m10s elapsed]
aws_instance.id01: Still destroying... [id=i-05bea27f12bdf00d7, 00m20s elapsed]
aws_instance.id01: Still destroying... [id=i-05bea27f12bdf00d7, 00m30s elapsed]
aws_instance.id01: Still destroying... [id=i-05bea27f12bdf00d7, 00m40s elapsed]
aws_instance.id01: Destruction complete after 50s

Destroy complete! Resources: 1 destroyed.
root@ip-172-31-8-242:/home/ubuntu/terraform_directory#

```